

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т

Тема задания: «ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»

Обучающийся: Данилова Айаана Васильевна, К3241

Санкт-Петербург, 2026

Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров.
2. Создать авторские процедуры.
3. Создать 7 триггеров по ИЗ.

Индивидуальный вариант работы

Вариант 13 - Система обслуживания в ресторане КлодМоне.

В ресторане пока предусмотрено 3 типа сотрудников, которые соответственно относятся каждый к одной из трех категорий: Зал, Кухня, Администрация. Паспортная информация всех работников хранится со статусом документа, чтобы отслеживать действующий. Должности из категории “Кухня” обязательно иметь мед. книжку, что фиксировано в сущности категории с помощью булевого флага. Предусмотрена кадровая история. Непосредственно основные типы сотрудников:

- официант, за которым закрепляются физические столы на определенной смене посредством добавления записи в таблицу закрепления
 - администратор, ответственный за смену
 - повар, готовящий блюда из заказа
- Есть таблица работника на смене, выступающий в качестве внешнего ключа в случаях выше.

Ресторан не специализируется на какой-то определенной кухне и имеет обычное и универсальное меню, разделенное по категориям.

Ингредиенты поставляются партиями от компаний-поставщиков и хранятся на складах. Бд подходит для одного филиала, но это не мешает пользоваться ею и крупным конгломератам с несколькими филиалами, так как для каждого филиала, бд можно поднять на одном из серверов или ноды кластера сервера.

Задание 4. Создать хранимые процедуры:

- ★ Вывести сведения о заказах заданного официанта на заданную дату.
- ★ Выполнить расчет стоимости заданного заказа.
- ★ Повышения оклада заданного сотрудника на 30 % при повышении его категории.

Задание 5. Создать необходимые триггеры.

1. Хранимые процедуры

1.1. Вывести сведения о заказах заданного официанта на заданную дату.

Задание требует вывести сведения о заказах, обслуженным за один день некоторым официантом, то есть большого количества строк, поэтому возвращать аргумент типа OUT не подходит, поэтому было реализовано 2 варианта:

- с RAISE NOTICE обычно использующегося для отладки.
- со строкой запроса копирования в файл, путь к которому также в качестве аргумента передается, внутри процедуры и ее выполнение через EXECUTE.

Скрипт первого варианта:

```
CloudMonet=# \sf get_waiter_orders_on_date
CREATE OR REPLACE PROCEDURE v1.get_waiter_orders_on_date(IN p_waiter_id
integer, IN p_shift_date date)
LANGUAGE plpgsql
SET search_path = v1
AS $procedure$
DECLARE
    r RECORD;
    v_found BOOLEAN := FALSE;
BEGIN
    RAISE NOTICE '--- Checking orders for Waiter % on Date: % ---',
p_waiter_id, p_shift_date;

    FOR r IN (
        SELECT
            o.order_id,
            o.created_at::time as order_time,
            sum(os.dish_count) as dishes_count,
            array_agg(d.dish_name) as scoup_list,
            sum(os.row_total) as order_total_cost
        FROM worker w
        JOIN worker_shift ws ON w.worker_id = ws.worker_id
        JOIN shift s ON s.shift_id = ws.shift_id
```

```

JOIN pinned_table pt ON ws.ws_id = pt.ws_id
JOIN "order" o ON o.pinning_id = pt.pinning_id
JOIN order_scoup os ON os.order_id = o.order_id
JOIN dish d on d.dish_id = os.dish_id
WHERE w.worker_id = p_waiter_id
      AND s.shift_start::date = p_shift_date
GROUP BY o.order_id, o.created_at
ORDER BY o.created_at
)
LOOP
    v_found := TRUE;
    RAISE NOTICE 'Order %: Time: %, Dishes count: %, Scoup: %, Total
cost: %',
                r.order_id, r.order_time, r.dishes_count,
r.scoup_list, r.order_total_cost;
END LOOP;

IF NOT v_found THEN
    RAISE NOTICE 'Orders not found for this criteria.';
END IF;
END;
$procedure$

```

Вывод командной строки:

```

CloudMonet=# call v1.get_waiter_orders_on_date(4202, '2026-04-28');
ЗАМЕЧАНИЕ: --- Checking orders for waiter 4202 on Date: 2026-04-28 ---
ЗАМЕЧАНИЕ: Order 9047: Time: 07:23:14.216356, Dishes count: 15, Scoup: {"Борщ с говядиной","Стейк Рибай","Цезарь с курицей","Двойной Эспрессо","Паста Карбонара",чизкейк}, Total cost: 6680.0000
ЗАМЕЧАНИЕ: Order 109254: Time: 12:02:21.609761, Dishes count: 13, Scoup: {"Двойной Эспрессо",чизкейк,"Паста Карбонара", "Стейк Рибай","Цезарь с курицей","Борщ с говядиной"}, Total cost: 7440.0000
ЗАМЕЧАНИЕ: Order 463058: Time: 02:20:46.50789, Dishes count: 13, Scoup: {"Борщ с говядиной","Двойной Эспрессо","Паста Карбонара", "Стейк Рибай","Цезарь с курицей",чизкейк}, Total cost: 7260.0000
ЗАМЕЧАНИЕ: Order 29778: Time: 17:26:30.589416, Dishes count: 10, Scoup: {"Стейк Рибай","Цезарь с курицей","Борщ с говядиной", "Паста Карбонара",чизкейк,"Двойной Эспрессо"}, Total cost: 5270.0000
ЗАМЕЧАНИЕ: Order 409588: Time: 12:53:29.209125, Dishes count: 14, Scoup: {"Паста Карбонара", "Двойной Эспрессо",чизкейк, "Борщ с говядиной", "Стейк Рибай", "Цезарь с курицей"}, Total cost: 7210.0000
ЗАМЕЧАНИЕ: Order 389878: Time: 07:09:36.327166, Dishes count: 12, Scoup: {"Стейк Рибай","Паста Карбонара", "Двойной Эспрессо", "Борщ с говядиной", чизкейк, "Цезарь с курицей"}, Total cost: 6170.0000
ЗАМЕЧАНИЕ: Order 229939: Time: 15:13:14.868882, Dishes count: 10, Scoup: {чизкейк, "Двойной Эспрессо", "Борщ с говядиной", "Паста Карбонара", "Цезарь с курицей", "Стейк Рибай"}, Total cost: 5970.0000
ЗАМЕЧАНИЕ: Order 115965: Time: 03:51:13.61357, Dishes count: 10, Scoup: {чизкейк, "Цезарь с курицей", "Двойной Эспрессо", "Паста Карбонара", "Стейк Рибай", "Борщ с говядиной"}, Total cost: 5090.0000
ЗАМЕЧАНИЕ: Order 141494: Time: 10:04:48.937504, Dishes count: 11, Scoup: {"Стейк Рибай", "Двойной Эспрессо", чизкейк, "Цезарь с курицей", "Паста Карбонара", "Борщ с говядиной"}, Total cost: 6910.0000
ЗАМЕЧАНИЕ: Order 221290: Time: 06:35:32.437926, Dishes count: 11, Scoup: {"Двойной Эспрессо", "Стейк Рибай", "Паста Карбонара", чизкейк, "Борщ с говядиной", "Цезарь с курицей"}, Total cost: 5410.0000
ЗАМЕЧАНИЕ: Order 364564: Time: 19:07:27.636344, Dishes count: 8, Scoup: {"Двойной Эспрессо", чизкейк, "Паста Карбонара", "Стейк Рибай", "Цезарь с курицей", "Борщ с говядиной"}, Total cost: 4810.0000
ЗАМЕЧАНИЕ: Order 405019: Time: 21:05:37.252023, Dishes count: 15, Scoup: {"Борщ с говядиной", "Стейк Рибай", "Двойной Эспрессо", чизкейк, "Цезарь с курицей", "Паста Карбонара"}, Total cost: 6710.0000
ЗАМЕЧАНИЕ: Order 471711: Time: 09:25:10.197499, Dishes count: 13, Scoup: {"Стейк Рибай", "Цезарь с курицей", "Борщ с говядиной", "Паста Карбонара", "Двойной Эспрессо", чизкейк}, Total cost: 6140.0000
ЗАМЕЧАНИЕ: Order 470976: Time: 03:53:57.243395, Dishes count: 12, Scoup: {"Стейк Рибай", "Цезарь с курицей", "Двойной Эспрессо", "Борщ с говядиной", "Паста Карбонара", чизкейк}, Total cost: 6680.0000
ЗАМЕЧАНИЕ: Order 263417: Time: 13:13:20.764011, Dishes count: 11, Scoup: {чизкейк, "Стейк Рибай", "Паста Карбонара", "Цезарь с курицей", "Двойной Эспрессо", "Борщ с говядиной"}

```

Скрипт второго варианта:

```

CloudMonet=# \sf get_waiter_orders_on_date_to_file
CREATE OR REPLACE PROCEDURE v1.get_waiter_orders_on_date_to_file(IN
p_waiter_id integer, IN p_shift_date date, IN p_file_path text)
LANGUAGE plpgsql
SET search_path = v1

```

```

AS $procedure$
BEGIN
    RAISE NOTICE 'Starting export for waiter % on date % to file %',
    p_waiter_id, p_shift_date, p_file_path;

    EXECUTE format('
        COPY (
            SELECT
                o.order_id,
                o.created_at::time as order_time,
                sum(os.dish_count) as total_dishes,
                array_agg(d.dish_name) as dishes_list,
                sum(os.row_total) as order_cost
            FROM worker w
            JOIN worker_shift ws ON w.worker_id = ws.worker_id
            JOIN shift s ON s.shift_id = ws.shift_id
            JOIN pinned_table pt ON ws.ws_id = pt.ws_id
            JOIN "order" o ON o.pinning_id = pt.pinning_id
            JOIN order_scoup os ON os.order_id = o.order_id
            JOIN dish d ON d.dish_id = os.dish_id
            WHERE w.worker_id = %L
                AND s.shift_start::date = %L
            GROUP BY o.order_id, o.created_at
            ORDER BY o.created_at
        ) TO %L WITH (FORMAT CSV, HEADER, ENCODING ''UTF8'')',
    p_waiter_id, p_shift_date, p_file_path);

    RAISE NOTICE 'Export completed successfully.';

EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Error during export: %', SQLERRM;
END;
$procedure$

```

Вывод командной строки:

```

CloudMonet=# CALL get_waiter_orders_on_date_to_file(
CloudMonet=#      4202,
CloudMonet=#      '2026-04-28',
CloudMonet=#      'C:/temp/waiter_report.csv'
CloudMonet=# );
ЗАМЕЧАНИЕ: Starting export for waiter 4202 on date 2026-04-28 to file C:/temp/waiter_report.csv
ЗАМЕЧАНИЕ: Export completed successfully.
CALL
CloudMonet=#

```

Содержимое файла:

```

order_id,order_time,total_dishes,dishes_list,order_cost|
9047,04:23:14.216356,15,{"Борщ с говядиной","Стейк Рибай","Цезарь с курицей","Двойной Эспрессо","Паста Карбонара","Чизкейк"},6680.0000
109254,09:02:21.609761,13,{"Двойной Эспрессо","Чизкейк","Паста Карбонара","Стейк Рибай","Цезарь с курицей","Борщ с говядиной"},7440.0000
463058,23:20:46.50789,13,{"Борщ с говядиной","Двойной Эспрессо","Паста Карбонара","Стейк Рибай","Цезарь с курицей","Чизкейк"},7260.0000
29778,14:26:30.589416,10,{"Стейк Рибай","Цезарь с курицей","Борщ с говядиной","Паста Карбонара","Чизкейк","Двойной Эспрессо"},5270.0000
409588,09:53:29.209125,14,{"Паста Карбонара","Двойной Эспрессо","Чизкейк","Борщ с говядиной","Стейк Рибай","Цезарь с курицей"},7210.0000
389878,04:09:36.327166,12,{"Стейк Рибай","Паста Карбонара","Двойной Эспрессо","Борщ с говядиной","Чизкейк","Цезарь с курицей"},6170.0000
229939,12:13:14.868882,10,{"Чизкейк","Двойной Эспрессо","Борщ с говядиной","Паста Карбонара","Цезарь с курицей","Стейк Рибай"},5970.0000
115965,00:51:13.61357,10,{"Чизкейк","Цезарь с курицей","Двойной Эспрессо","Паста Карбонара","Стейк Рибай","Борщ с говядиной"},5090.0000
141494,07:04:48.937504,11,{"Стейк Рибай","Двойной Эспрессо","Чизкейк","Цезарь с курицей","Паста Карбонара","Борщ с говядиной"},7440.0000

```

Строка 1, столбец 56 | 46 974 символа | Обычный текст | 100% | Windows (CRLF) | UTF-8

1.2. Выполнить расчет стоимости заданного заказа

Скрипт:

```

CloudMonet=# \sf v1.get_order_total_cost
CREATE OR REPLACE PROCEDURE v1.get_order_total_cost(IN p_order_id
integer, OUT p_total_cost numeric)
LANGUAGE plpgsql
SET search_path = v1
AS $procedure$
BEGIN
    SELECT sum(row_total)
    INTO p_total_cost
    FROM v1.order_scoup
    WHERE order_id = p_order_id AND status != 'cancelled';

    IF p_total_cost IS NULL THEN
        RAISE NOTICE 'Warning: Order % not found. Setting cost to 0.',
p_order_id;
        p_total_cost := 0;
    END IF;
END;
$procedure$

```

Вывод:

```

CloudMonet=# call get_order_total_cost(341439, null);
 p_total_cost
-----
      6350.0000
(1 строка)

```

1.3. Повышения оклада заданного сотрудника на 30 % при повышении его категории

Будем повышать официанта стажера.

```
CloudMonet=# select * from worker where position_id = 11 order by random() limit 1;
```

worker_id	full_name	timesheet_num	is_active	position_id	phone	email	salary
4654	Макаров Роман	10454	t	11	+73127665906	6985@inbox.ru	30000.0000

(1 строка)

Наличие повышения категории будет подтверждаться записью в кадровой истории с типом события promotion. Процедура изменит колонку с окладом, то есть salary, только если он не был уже повышен в промежутке времени после повышения и до текущего момента, то есть при отсутствии записи типа salary_up, и существования исходного значения salary, в ином случае процедура оповестит о неполадках или упадет с ошибкой. После успешного увеличения добавляется запись в кадровую историю о повышении оклада согласно шаблону.

Скрипт:

```
CloudMonet=# \sf v1.apply_salary_upgrade
CREATE OR REPLACE PROCEDURE v1.apply_salary_upgrade(IN p_worker_id
integer)
LANGUAGE plpgsql
AS $procedure$
DECLARE
    v_last_promotion date;
    v_last_salary_up date;
    v_cur_salary NUMERIC;
    v_new_salary NUMERIC;
BEGIN
    select salary
    into v_cur_salary
    from v1.worker
    where worker_id = p_worker_id
    for update;

    if not found then
        raise exception 'Сотрудник % не существует.', p_worker_id;
    end if;

    SELECT event_date INTO v_last_promotion
    FROM v1.career_log
    WHERE worker_id = p_worker_id AND event_type = 'promotion'
    ORDER BY event_date DESC LIMIT 1;
```

```

SELECT event_date INTO v_last_salary_up
FROM v1.career_log
WHERE worker_id = p_worker_id AND event_type = 'salary_up'
ORDER BY event_date DESC LIMIT 1;

IF v_last_promotion IS NULL THEN
    RAISE EXCEPTION 'Неполадка: Запись о повышении категории для
сотрудника % не найдена.', p_worker_id;

    ELSIF v_last_salary_up IS NOT NULL AND v_last_salary_up >
v_last_promotion THEN
    RAISE NOTICE 'Уведомление: Оклад сотрудника % уже был повышен
после последнего повышения категории.', p_worker_id;

    ELSIF v_cur_salary IS NULL OR v_cur_salary <= 0 THEN
    RAISE EXCEPTION 'Неполадка: Исходный оклад сотрудника % равен
NULL или 0.', p_worker_id;

ELSE
    v_new_salary := round(v_cur_salary * 1.30, 4);

    UPDATE v1.worker
    SET salary = v_new_salary
    WHERE worker_id = p_worker_id;

    INSERT INTO v1.career_log (worker_id, event_type, new_val,
description)
    VALUES (
        p_worker_id,
        'salary_up',
        v_new_salary::text,
        'Автоматическое повышение оклада на 30% в связи с ростом
категории'
    );

    RAISE NOTICE 'Успех: Оклад сотрудника % повышен до %.',
p_worker_id, v_new_salary;
END IF;
END;
$procedure$

```

Результат:


```
CloudMonet=# call v1.apply_salary_upgrade(4654);
ЗАМЕЧАНИЕ:  Успех: Оклад сотрудника 4654 повышен до 39000.000000.
CALL
```

Обновленная зарплата в таблице сотрудников:

```
CloudMonet=# select * from worker where worker_id = 4654;
worker_id | full_name | timesheet_num | is_active | position_id | phone | email | salary
-----+-----+-----+-----+-----+-----+-----+-----
4654 | Макаров Роман | 10454 | t | 11 | +73127665906 | 6985@inbox.ru | 39000.0000
(1 строка)
```

Добавленная запись в кадровую историю:

```
CloudMonet=# select * from career_log where worker_id = 4654;
note_id | worker_id | event_type | new_val | description
-----+-----+-----+-----+-----
1950 | 1956 | 4654 | REPRIMAND | 2025-12-06 | Выговор | Нарушение трудовой дисциплины или регламента
2550 | 18444 | 4654 | COMMENDATION | 2023-12-03 | Благодарность | Поощрение за высокие показатели работы
3225 | 3225 | 4654 | PROMOTION | 10 | Повышение должности: Официант (стажер) -> Официант
3226 | 3226 | 4654 | SALARY_UP | 2026-05-12 | Автоматическое повышение оклада на 30% в связи с ростом категории
(4 строки)
```

При повторной попытке повышения:

```
CloudMonet=# call v1.apply_salary_upgrade(4654);
ЗАМЕЧАНИЕ:  Уведомление: Оклад сотрудника 4654 уже был повышен ранее.
CALL
```

2. Авторские процедуры

2.1. Изменение статуса сотрудника при наличии записи в кадровой истории об отпуске, больничном, увольнении.

Скрипт:

```
CloudMonet=# \sf apply_worker_deactive
CREATE OR REPLACE PROCEDURE v1.apply_worker_deactive(IN p_worker_id
integer)
LANGUAGE plpgsql
SET search_path TO 'v1'
AS $procedure$
declare
    v_is_active    boolean;
    v_term_date    timestamp;
    v_vac_date     timestamp;
    v_sick_date    timestamp;
    v_reinst_date  timestamp;
```

```

    v_ret_vac_date timestamp;
    v_ret_sick_date timestamp;
begin

    select is_active into v_is_active
    from v1.worker where worker_id = p_worker_id
    for update;

    if not found then
        raise exception 'Ошибка: Сотрудник с ID % не найден',
p_worker_id;
    end if;

    select
        max(event_date) filter (where event_type = 'termination'),
        max(event_date) filter (where event_type = 'vacation'),
        max(event_date) filter (where event_type = 'sick_leave'),
        max(event_date) filter (where event_type = 'reinstatement'),
        max(event_date) filter (where event_type =
'return_from_vacation'),
        max(event_date) filter (where event_type =
'return_from_sick_leave')
    into
        v_term_date, v_vac_date, v_sick_date,
        v_reinst_date, v_ret_vac_date, v_ret_sick_date
    from v1.career_log
    where worker_id = p_worker_id;

    if v_term_date is not null and v_term_date > coalesce(v_reinst_date,
'1900-01-01') then
        if v_is_active then
            update v1.worker set is_active = false where worker_id =
p_worker_id;
            raise notice 'Сотрудник % деактивирован (увольнение от %)',
p_worker_id, v_term_date;
        end if;
        return;
    end if;

    if v_vac_date is not null and v_vac_date > coalesce(v_ret_vac_date,
'1900-01-01') then
        if v_is_active then
            update v1.worker set is_active = false where worker_id =
p_worker_id;
            raise notice 'Сотрудник % деактивирован (отпуск от %)',
p_worker_id, v_vac_date;

```

```

        end if;
        return;
    end if;

    if v_sick_date is not null and v_sick_date >
    coalesce(v_ret_sick_date, '1900-01-01') then
        if v_is_active then
            update v1.worker set is_active = false where worker_id =
p_worker_id;
            raise notice 'Сотрудник % деактивирован (больничный от %)',
p_worker_id, v_sick_date;
        end if;
        return;
    end if;

    if not v_is_active then
        update v1.worker set is_active = true where worker_id =
p_worker_id;
        raise notice 'Сотрудник % активирован (все события ухода закрыты
возвратами)', p_worker_id;
    else
        raise notice 'Изменений не требуется: сотрудник % активен и не
имеет открытых пропусков.', p_worker_id;
    end if;

end;
$procedure$

```

Будем увольнять работника с идентификатором 10845:

```

CloudMonet=# select * from worker w where not exists(select 1 from worker_shift ws where ws.worker_id = w.worker_id) ord
er by random() limit 1;
 worker_id | full_name | timesheet_num | is_active | position_id | phone | email | sa
lary
-----+-----+-----+-----+-----+-----+-----+-----
10845 | Неактивный Тест 41 | 10642 | t | 1 | +790000000041 | test_worker_41@itmo.ru | 1500
00.0000
(1 строка)

```

Вывод процедуры об успешной деактивации при наличии записи об увольнении:

```

CloudMonet=# insert into career_log(worker_id, event_type) values (10845, 'termination');
INSERT 0 1
CloudMonet=# call apply_worker_deactive(10845);
ЗАМЕЧАНИЕ: Сотрудник 10845 деактивирован (увольнение от 2026-05-13 18:19:34.971264)
CALL

```

Процедура ничего не вывела и не изменила, а просто вернула null при повторном применении:

```
CloudMonet=# call apply_worker_deactive(10845);  
CALL
```

После добавления записи о восстановлении:

```
CloudMonet=# insert into career_log(worker_id, event_type) values (10845, 'reinstatement');  
INSERT 0 1  
CloudMonet=# call apply_worker_deactive(10845);  
ЗАМЕЧАНИЕ: Сотрудник 10845 активирован (все события ухода закрыты возвратами)  
CALL
```

При повторном выполнении при наличии и увольнения и восстановления в корректной последовательности(первое не позже второго по времени произошло, именно по времени timestamp, не айди и т.д.):

```
CloudMonet=# call apply_worker_deactive(10845);  
ЗАМЕЧАНИЕ: Изменений не требуется: сотрудник 10845 активен и не имеет открытых пропусков.  
CALL
```

При повторном увольнении и при наличии получается неактуальной уже записи о восстановлении процедура корректно модифицирует статус работника:

```
CloudMonet=# insert into career_log(worker_id, event_type) values (10845, 'termination');  
INSERT 0 1  
CloudMonet=# call apply_worker_deactive(10845);  
ЗАМЕЧАНИЕ: Сотрудник 10845 деактивирован (увольнение от 2026-05-13 18:32:17.459382)  
CALL
```

2.2. Динамический подсчет реальной стоимости ингредиента на основе текущей партии срок годности которой выйдет скорее всех остальных

Скрипт:

```
CloudMonet=# \sf refresh_ingredient_unit_cost  
CREATE OR REPLACE PROCEDURE v1.refresh_ingredient_unit_cost(IN p_ingr_id  
integer)  
LANGUAGE plpgsql  
SET search_path TO 'v1'  
AS $procedure$  
declare  
    v_ingr_exists    boolean;  
    v_batch_id       integer;  
    v_unit_cost      numeric(19,4);  
    v_exp_date       date;  
    v_scoop_status   character varying(9);
```

```

        v_curr_qty          numeric(12,3);
begin
    select true into v_ingr_exists
    from v1.ingredient
    where ingredient_id = p_ingr_id
    for update;

    if not found then
        raise exception 'Ошибка: Ингредиент с ID % не найден',
p_ingr_id;
    end if;

    select
        b.batch_id,
        b.ingr_unit_cost,
        b.batch_exp_data,
        ss.st_sc_status,
        ss.st_sc_curr_qty
    into
        v_batch_id,
        v_unit_cost,
        v_exp_date,
        v_scoup_status,
        v_curr_qty
    from v1.batch b
    join v1.storage_scoup ss
        on ss.batch_id = b.batch_id
    where b.ingr_id = p_ingr_id
        and ss.st_sc_status = 'available'
        and ss.st_sc_curr_qty > 0
        and b.batch_exp_data >= current_date
    order by b.batch_exp_data asc
    limit 1
    for update of b, ss;

    if not found then
        raise notice 'Ингредиент %: нет доступных партий -- стоимость
обнуляется', p_ingr_id;

        update v1.ingredient
        set ingr_unit_cost = 0
        where ingredient_id = p_ingr_id;

        return;
    end if;

```

```

        raise notice
            'Ингредиент %: актуальная партия % (годна до: %, qty: %, цена:
            %)',
            p_ingr_id, v_batch_id, v_exp_date, v_curr_qty, v_unit_cost;

        update v1.ingredient
        set ingr_unit_cost = v_unit_cost
        where ingredient_id = p_ingr_id;

exception
    when deadlock_detected then
        raise exception 'Ошибка: дедлок при обработке ингредиента %.
        Повторите запрос.', p_ingr_id;

    when lock_not_available then
        raise exception 'Ошибка: строка ингредиента % заблокирована
        другой транзакцией.', p_ingr_id;

    when others then
        raise exception 'Непредвиденная ошибка при обработке ингредиента
        %: % (SQLSTATE: %)',
            p_ingr_id, sqlerrm, sqlstate;
end;
$procedure$

```

Применение:

```

CloudMonet=# call refresh_ingredient_unit_cost(1);
ЗАМЕЧАНИЕ: Ингредиент 1: актуальная партия 4650 (годна до: 2026-05-13, qty: 6.593, цена: 113.9543)
CALL

```

Несуществующий ингредиент:

```

CloudMonet=# call refresh_ingredient_unit_cost(100);
ОШИБКА: Непредвиденная ошибка при обработке ингредиента 100: Ошибка: Ингредиент с ID 100 не найден (SQLSTATE: P0001)
КОНТЕКСТ: функция PL/pgSQL refresh_ingredient_unit_cost(integer), строка 68, оператор RAISE

```

Ингредиент, которого нет на складе:

```

CloudMonet=# call refresh_ingredient_unit_cost(100073);
ЗАМЕЧАНИЕ: Ингредиент 100073: нет доступных партий – стоимость обнуляется
CALL

```

3. Триггеры

3.1. Медкнижки

Для таблицы целостность данных защищена ограничениями и частичным уникальным индексом, который гарантирует, что у работника может быть строго одна активная запись.

При этом признак **is_active** отражает управленческий статус документа (то, чем сейчас располагает администрация), а не его временную валидность (срок годности). Документ может иметь статус активного, даже если срок его действия уже истек.

```
Индексы:
"pk_med_book" PRIMARY KEY, btree (med_book_id)
"idx_med_book_expires" btree (expires_date)
"idx_med_book_worker" btree (worker_id)
"uq_med_book_single_active" UNIQUE, btree (worker_id) WHERE is_active = true
Ограничения-проверки:
"chk_date_interval" CHECK (expires_date < (issued_date + '2 years'::interval))
"chk_med_book_dates" CHECK (expires_date > issued_date)
Ограничения внешнего ключа:
"fk_med_book_worker" FOREIGN KEY (worker_id) REFERENCES worker(worker_id) ON UPDATE CASCADE ON DELETE RESTRICT
```

Скрипт триггера:

```
CloudMonet=# \sf trg_med_book
CREATE OR REPLACE FUNCTION v1.trg_med_book()
  RETURNS trigger
  LANGUAGE plpgsql
  SET search_path TO 'v1'
  AS $function$
BEGIN

  IF TG_OP = 'INSERT' THEN

    IF NEW.issued_date > CURRENT_DATE THEN
      RAISE EXCEPTION 'ERROR: Document issue date cannot be in the
future.'
        USING ERRCODE = 'P0011';
    END IF;

    IF NEW.is_active = true AND NEW.expires_date < CURRENT_DATE THEN
      RAISE EXCEPTION 'ERROR: Cannot activate an expired
document.'
        USING ERRCODE = 'P0004';
    END IF;

    RETURN NEW;
```

```

        ELSIF TG_OP = 'UPDATE' THEN

            IF NEW.issued_date != OLD.issued_date AND NEW.issued_date >
CURRENT_DATE THEN
                RAISE EXCEPTION 'ERROR: Document issue date cannot be in the
future.'
                    USING ERRCODE = 'P0011';
            END IF;

            IF NEW.is_active = true THEN
                IF NEW.expires_date < CURRENT_DATE THEN
                    RAISE EXCEPTION 'ERROR: Cannot activate an expired
document.'
                        USING ERRCODE = 'P0004';
                END IF;
            END IF;

            IF OLD.is_active = true AND (NEW.issued_date != OLD.issued_date
OR NEW.expires_date != OLD.expires_date) THEN
                IF NEW.expires_date < CURRENT_DATE THEN
                    RAISE EXCEPTION 'ERROR: Cannot update dates of an active
document to an expired period.'
                        USING ERRCODE = 'P0005';
                END IF;
            END IF;

            RETURN NEW;

        ELSIF TG_OP = 'DELETE' THEN

            IF OLD.is_active = true THEN
                RAISE EXCEPTION 'ERROR: Deletion of an active document is
strictly prohibited.'
                    USING ERRCODE = 'P0003';
            END IF;

            RETURN OLD;

        END IF;

        RETURN NULL;
    END;
$function$;

```



```
create or replace trigger trg_med_book_check
before insert or update or delete on med_book
for each row execute function trg_med_book();
```

Ниже представлены авторские процедуры, реализующие защищенный слой API непосредственно внутри базы данных. Бэкенд-приложение взаимодействует с реестром медицинских книжек исключительно через эти ручки, что гарантирует атомарность операций и инкапсулирует сложные сценарии обновления данных.

Авторская процедура №1. Замена медкнижки на новую (Добавление новой + гашение старой)

Проблема: Бэк пытается вставить новую активную книжку, но падает, так как уникальный индекс не позволяет иметь две активные записи.

Решение: Процедура, которая делает это атомарно (в одной транзакции): сначала безопасно гасит старую запись, а затем вставляет новую.

```
CloudMonet=# \sf add_new_med_book
CREATE OR REPLACE PROCEDURE v1.add_new_med_book(IN p_worker_id integer,
IN p_issued_date date, IN p_expires_date date)
LANGUAGE plpgsql
SET search_path TO 'v1'
AS $procedure$
begin
    UPDATE v1.med_book
    SET is_active = false
    WHERE worker_id = p_worker_id AND is_active = true;

    INSERT INTO v1.med_book (worker_id, issued_date, expires_date,
is_active)
    VALUES (p_worker_id, p_issued_date, p_expires_date, true);

    RAISE INFO 'INFO: Medical book for worker_id % has been successfully
updated.', p_worker_id;
end;
$procedure$
```

Авторская процедура №2. Безопасное удаление / Увольнение сотрудника (Мягкое удаление)

Проблема: Медкнижку активного сотрудника нельзя удалить (срабатывает триггер P0003), но админу или бэку кровь из носа нужно её убрать (например, запись завели по ошибке, или сотрудник уволился).

Решение: Вместо физического DELETE (который уничтожает историю и заблокирован триггером), используется процедура деактивации. Физическое удаление разрешается только после деактивации.

```
CREATE OR REPLACE PROCEDURE v1.archive_med_book(  
    p_med_book_id integer  
)  
LANGUAGE plpgsql  
AS $procedure$  
declare  
    v_is_active boolean;  
begin  
    SELECT is_active INTO v_is_active FROM v1.med_book WHERE med_book_id  
= p_med_book_id;  
  
    if v_is_active IS NULL then  
        RAISE EXCEPTION 'ERROR: Medical book record with ID % not  
found.', p_med_book_id;  
    end if;  
  
    UPDATE v1.med_book  
    SET is_active = false  
    WHERE med_book_id = p_med_book_id;  
  
    RAISE INFO 'INFO: Medical book with ID % has been successfully  
archived.', p_med_book_id;  
end;  
$procedure$;
```

Авторская процедура №3. Полное удаление ошибочной записи (Если РЕАЛЬНО надо сделать DELETE)

Проблема: Вы случайно опечатались в данных и хотите физически удалить строку через DELETE, но триггер кричит: «Удаление запрещено!».

Решение: Процедура-обертка, которая сначала снимает флаг активности, а

затем удаляет строку. Всё происходит внутри одной транзакции, поэтому триггер останется доволен (он увидит **DELETE** уже неактивной строки).

```
CREATE OR REPLACE PROCEDURE v1.force_delete_med_book(  
    p_med_book_id integer  
)  
SET search_path TO 'v1'  
LANGUAGE plpgsql  
AS $procedure$  
begin  
    UPDATE v1.med_book  
    SET is_active = false  
    WHERE med_book_id = p_med_book_id;  
  
    DELETE FROM v1.med_book  
    WHERE med_book_id = p_med_book_id;  
  
    RAISE INFO 'INFO: Medical book record with ID % has been physically  
deleted from the system.', p_med_book_id;  
end;  
$procedure$;
```

У всех сотрудников по одной активной медкнижке:

```
CloudMonet=# select worker_id from (select m.worker_id, count(*) over (partition by m.worker_id order by m.expires_date)  
as active_count from med_book m where m.is_active = true) where active_count > 1;  
worker_id  
-----  
(0 строк)
```

Будем работать с работником с идентификатором 4201:

```
CloudMonet=# select * from med_book where worker_id = 4201;  
 med_book_id | worker_id | issued_date | expires_date | is_active | notes  
-----+-----+-----+-----+-----+-----  
          71554 |         4201 | 2026-05-18 | 2027-05-18 | f         |  
          71558 |         4201 | 2026-05-18 | 2027-05-18 | t         |  
          65648 |         4201 | 2026-04-15 | 2027-04-25 | f         |  
(3 строки)
```

Проброс ошибки при добавлении медкнижки с будущей датой выдачи с помощью процедуры `add_new_med_book`:

```
CloudMonet=# CALL v1.add_new_med_book(4201, '2027-05-18', '2028-05-18');  
ОШИБКА: ERROR: Cannot update a medical book with a future issue date (Issued: 2027-05-18, Current date: 2026-05-18)  
КОНТЕКСТ: функция PL/pgSQL trg_med_book(), строка 23, оператор RAISE  
SQL-оператор: "UPDATE v1.med_book  
SET is_active = false  
WHERE worker_id = p_worker_id AND is_active = true"  
функция PL/pgSQL add_new_med_book(integer,date,date), строка 3, оператор SQL-оператор
```

Добавление медкнижки с правильной датой выдачи:

```
CloudMonet=# CALL v1.add_new_med_book(4201, '2026-05-18'::date, '2027-05-18'::date);
ИНФОРМАЦИЯ: INFO: Medical book for worker_id 4201 has been successfully updated.
CALL
CloudMonet=# select * from med_book where worker_id = 4201;
 med_book_id | worker_id | issued_date | expires_date | is_active | notes
-----+-----+-----+-----+-----+-----
          71560 |         4201 | 2027-05-18 | 2028-05-18 | f         |
          71561 |         4201 | 2026-05-18 | 2027-05-18 | t         |
          71554 |         4201 | 2026-05-18 | 2027-05-18 | f         |
          71558 |         4201 | 2026-05-18 | 2027-05-18 | f         |
          65648 |         4201 | 2026-04-15 | 2027-04-25 | f         |
(5 строк)
```

Принудительное удаление неправильной записи с будущей датой выдачи которая была случайно добавлена, когда процедура некорректно работала:

```
CloudMonet=# call force_delete_med_book(71560);
ИНФОРМАЦИЯ: INFO: Medical book record with ID 71560 has been physically deleted from the system.
CALL
```

Проброс ошибки при попытке удалить актуальную медкнижку:

```
CloudMonet=# call force_delete_med_book(71562);
ОШИБКА: ERROR: Cannot force delete an ACTIVE medical book. Archive it first using archive_med_book().
КОНТЕКСТ: функция PL/pgSQL force_delete_med_book(integer), строка 14, оператор RAISE
```

Успешная деактивация активной медкнижки:

```
CloudMonet=# call archive_med_book(71562);
ИНФОРМАЦИЯ: INFO: Medical book with ID 71562 has been successfully archived.
CALL
```

Добавление новой медкнижки при отсутствии записи для гашения:

```
CloudMonet=# CALL v1.add_new_med_book(4201, '2026-05-18'::date, '2027-05-18'::date);
ИНФОРМАЦИЯ: INFO: Medical book for worker_id 4201 has been successfully updated.
CALL
```

3.2. Паспорта

Ограничения и индексы таблицы паспортов сотрудников:

```
Индексы:
"pk_passport" PRIMARY KEY, btree (passport_id)
"uq_passport_single_active" UNIQUE, btree (worker_id) WHERE is_active = true
Ограничения-проверки:
"chk_department_code" CHECK (dep_code::text ~ '^d{3}-d{3}$'::text)
"chk_issued_before_expired_date" CHECK (issued_date < expires_date)
"chk_passport_num" CHECK (passport_num::text ~ '^d{6}$'::text)
"chk_passport_series" CHECK (passport_series::text ~ '^d{4}$'::text)
Ограничения внешнего ключа:
"fk_worker" FOREIGN KEY (worker_id) REFERENCES worker(worker_id)
```

Скрипт триггера:

```
CREATE OR REPLACE FUNCTION v1.trg_passport()
  RETURNS trigger
  LANGUAGE plpgsql
  SET search_path TO 'v1'
```

```
AS $function$

begin
    if TG_OP in ('INSERT', 'UPDATE') then

        if new.passport_num is not null then

            new.passport_num = regexp_replace(new.passport_num, '\s+',
'', 'g');

            end if;

            if new.passport_series is not null then

                new.passport_series = regexp_replace(new.passport_series,
'\s+', '', 'g');

                end if;

                if new.dep_code is not null then

                    new.dep_code = regexp_replace(new.dep_code, '\s+', '', 'g');

                    end if;

                    if new.issued_by is not null then

                        new.issued_by = trim(regexp_replace(new.issued_by, '\s+', '
', 'g'));

                        end if;

                        if new.reg_address is not null then

                            new.reg_address = trim(regexp_replace(new.reg_address,
'\s+', ' ', 'g'));

                            end if;

                            end if;

                            if TG_OP = 'INSERT' then

                                if new.is_active = true and new.expires_date < current_date then
```

```

        raise exception 'ERROR: Cannot insert an active passport
that has already expired (Expires: %, Current date: %)',

        new.expires_date, current_date

        using errcode = 'P0004';

    end if;
    return new;

elsif TG_OP = 'UPDATE' then

    if new.is_active = true then

        if new.expires_date < current_date then

            raise exception 'ERROR: Cannot activate an expired
passport (Expires: %, Current date: %)',

            new.expires_date, current_date

            using errcode = 'P0004';

        end if;

    end if;

    if old.is_active = true and (new.issued_date != old.issued_date
or new.expires_date != old.expires_date) then

        if new.expires_date < current_date then

            raise exception 'ERROR: Cannot update dates of an active
passport to an expired period'

            using errcode = 'P0005';

        end if;

    end if;

    return new;

elsif TG_OP = 'DELETE' then

    if old.is_active = true then

```

```

        raise exception 'ERROR: Deletion of an active passport is
strictly prohibited'
        using errcode = 'P0003';
    end if;

    return old;
end if;

return null;
end;
$function$;

CREATE OR REPLACE trg_passport_clean_and_check
BEFORE INSERT OR DELETE OR UPDATE ON passport
FOR EACH ROW EXECUTE FUNCTION trg_passport();

```

Будем работать с работником с ID 4321, с просроченным паспортом со статусом актуальности false.

При попытке вернуть актуальность вылетает ошибка о том, что просроченный паспорт невозможно сделать актуальным, то есть бизнес-логика подразумевает, что после изменения статуса у документа на не отслеживаемый, то сразу на замену необходимо добавить запись:

```

CloudMonet=# update passport set is_active = true where worker_id = 4321;
ОШИБКА:  ERROR: Cannot activate an expired medical book (Expires: 2026-05-17, Current date: 2026-05-19)
КОНТЕКСТ:  функция PL/pgSQL trg_passport(), строка 40, оператор RAISE
CloudMonet=#

```

Триггер автоматически удаляет пробелы в данных для вставки:

Строка из скриншота:

```

insert into passport(worker_id, reg_address, issued_by, issued_date,
expires_date, dep_code, passport_series, passport_num) values (4321, 'г.
Казань, ул. Лесная, д.126, кв.38', 'ОУФМС России по г.
Санкт-Петербургу', '2026-05-19', '2030-05-19', '437-59 8', '0123  ', '1
0 10 10 ');

```

```

CloudMonet=# alter table passport alter column is_active set default true;
ALTER TABLE
CloudMonet=# insert into passport(worker_id, reg_address, issued_by, issued_date, expires_date, dep_code, passport_series, passport_num) values (4321, 'г. Казань, ул. Лесная, д.126, кв.38', 'ОУФМС России по г. Санкт-Петербургу', '2026-05-19', '2030-05-19', '437-59 8', '0123  ', '1 0 10 10 ');
INSERT 0 1
CloudMonet=#

```

```

CloudMonet=# select * from passport where worker_id = 4321;
 passport_id | worker_id | passport_num | passport_series | issued_by | issued_date | dep_code | reg_address | expires_date | is_active |
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
 4805       |      4321 |    101010    |      0123       | ОУФМС России по г. Санкт-Петербургу | 2026-05-19 | 437-598 | г. Казань, ул. Лесная, д.126, кв.38 | 2030-05-19 | t         |
 4321       |      4321 |    534851    |      5209       | ОУФМС России по г. Санкт-Петербургу | 2022-05-17 | 437-598 | г. Казань, ул. Лесная, д.126, кв.38 | 2026-05-17 | f         |
(2 строки)

```

Если опечатались и при изменении департамента кода, также автоматически удаляются триггером пробелы:

```
CloudMonet=# update passport set dep_code = '437-59 6' where passport_id = 4805;
UPDATE 1
CloudMonet=# select * from passport where worker_id = 4321;
passport_id | worker_id | passport_num | passport_series | issued_by | issued_date | dep_code | reg_address | expires_date | is_active |
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4805 | 4321 | 101010 | 0123 | ОУФМС России по г. Санкт-Петербургу | 2026-05-19 | 437-596 | г. Казань, ул. Лесная, д.126, кв.38 | 2030-05-19 | t |
4321 | 4321 | 534851 | 5209 | ОУФМС России по г. Санкт-Петербургу | 2022-05-17 | 437-598 | г. Казань, ул. Лесная, д.126, кв.38 | 2026-05-17 | f |
(2 строки)
```

В остальном логика триггера идентична триггеру медицинских книжек. На уровне СУБД контролируется только валидность дат и автоматическое переключение статуса актуальности (**is_active**). Проверка и форматирование остальных текстовых полей делегированы на уровень бизнес-логики приложения (бэкенд). Это обусловлено высокой вариативностью сценариев модификации данных, которые нецелесообразно перегружать на уровне триггерных функций базы данных

3.3. Смена

В архитектуру базы данных были добавлены две таблицы-справочника:

- **shift_policy** для управления временными рамками типов смен
- **shift_status_setting**, использующая строгую типизацию на базе системного перечисления **ENUM**.

Данная таблица выступает в роли декларативной машины состояний (Finite State Machine), где поведение каждого статуса (возможность удаления смены, требования к проверке временных рамок и массив разрешенных векторов дальнейшего перехода) полностью инкапсулировано в метаданных.

Такое решение позволило полностью очистить код триггеров от хардкода и перенести управление бизнес-логикой жизненного цикла смен на уровень конфигурации данных.

Таблица **shift_policy**:

```
CloudMonet=# select * from shift_policy;
id | type_code | name_ru | standard_start_time | allowed_start_window | duration | is_active |
---+---+---+---+---+---+---+
1 | day | Дневная смена | 12:00:00 | 02:00:00 | 09:00:00 | t |
2 | night | Ночная смена | 21:00:00 | 02:00:00 | 12:00:00 | t |
(2 строки)
```



```

Индексы:
    "shift_policy_pkey" PRIMARY KEY, btree (id)
    "shift_policy_shift_type_code_key" UNIQUE CONSTRAINT, btree (type_code)
Ограничения-проверки:
    "chk_policy_allowed_window_positive" CHECK (allowed_start_window >= '00:00:00'::interval AND allowed_start_window <= '05:00:00'::interval)
    "chk_policy_duration_reasonable" CHECK (duration > '00:00:00'::interval AND duration <= '24:00:00'::interval)
Ссылки извне:
    TABLE "shift" CONSTRAINT "fk_shift_policy" FOREIGN KEY (type_id) REFERENCES shift_policy(id) ON UPDATE CASCADE ON DELETE RESTRICT
Метод доступа: heap

```

Таблица `shift_status_setting`:

```

CloudMonet=# select * from shift_status_setting;
 id | status_code | name_en | name_ru | is_deletable | requires_active_time | allowed_next_status_ids | is_active
-----+-----+-----+-----+-----+-----+-----+-----
  4 | closed      | Closed  | Закрыта | f             | f                     |                          | t
  5 | cancelled   | Cancelled | Отменена | t             | f                     |                          | t
  1 | ordered     | Booked   | Забронирована | t             | f                     | {2}                     | t
  2 | waiting     | Waiting  | В ожидании | t             | f                     | {3,5}                   | t
  3 | in-progress | In Progress | В процессе | f             | t                     | {4,5}                   | t
(5 строк)

```

```

Индексы:
    "shift_status_setting_pkey" PRIMARY KEY, btree (id)
    "uidx_active_shift_status_code" UNIQUE, btree (status_code) WHERE is_active = true
Ссылки извне:
    TABLE "shift" CONSTRAINT "fk_shift_status_setting" FOREIGN KEY (status_id) REFERENCES shift_status_setting(id) ON UPDATE CASCADE ON DELETE RESTRICT
Метод доступа: heap

```

Скрипт триггера:

```

CREATE OR REPLACE FUNCTION v1.trg_shift()
    RETURNS trigger
    LANGUAGE plpgsql
    SET search_path TO 'v1'
AS $function$
DECLARE
    v_start TIME WITHOUT TIME ZONE;
    v_interval INTERVAL;
    v_duration INTERVAL;
    v_fact_start_time TIME;
    v_is_deletable BOOLEAN;
    v_requires_time BOOLEAN;
    v_status_active BOOLEAN;
BEGIN
    IF TG_OP IN ('INSERT', 'UPDATE') THEN

        SELECT standard_start_time, allowed_start_window, duration
        INTO v_start, v_interval, v_duration
        FROM v1.shift_policy
        WHERE id = NEW.type_id AND is_active = true;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'ERROR: Policy identifier not found in
configuration.'
            USING ERRCODE = 'P0006';

```

```

END IF;

v_fact_start_time := NEW.shift_start::time;

IF v_fact_start_time < (v_start - v_interval) OR
v_fact_start_time > (v_start + v_interval) THEN
    RAISE EXCEPTION 'ERROR: Shift start time is outside the
allowed scheduling window.'
    USING ERRCODE = 'P0007';
END IF;

IF NEW.shift_end IS NULL THEN
    NEW.shift_end := NEW.shift_start + v_duration;
END IF;

SELECT requires_active_time, is_active
INTO v_requires_time, v_status_active
FROM v1.shift_status_setting WHERE id = NEW.status_id;

IF NOT FOUND OR v_status_active = false THEN
    RAISE EXCEPTION 'ERROR: Requested status change is blocked
by lifecycle rules.'
    USING ERRCODE = 'P0008';
END IF;

IF TG_OP = 'UPDATE' AND OLD.status_id <> NEW.status_id THEN
    IF NOT EXISTS (
        SELECT 1
        FROM v1.shift_status_setting
        WHERE id = OLD.status_id AND NEW.status_id =
ANY(allowed_next_status_ids)
    ) THEN
        RAISE EXCEPTION 'ERROR: Requested status change is
blocked by lifecycle rules.'
        USING ERRCODE = 'P0008';
    END IF;
END IF;

IF v_requires_time AND NOT (CLOCK_TIMESTAMP() BETWEEN
NEW.shift_start AND NEW.shift_end) THEN
    RAISE EXCEPTION 'ERROR: Cannot change status because current
real time is outside shift boundaries.'
    USING ERRCODE = 'P0009';
END IF;

RETURN NEW;

```

```

END IF;

IF TG_OP = 'DELETE' THEN
    SELECT is_deletable INTO v_is_deletable
    FROM v1.shift_status_setting
    WHERE id = OLD.status_id;

    IF NOT COALESCE(v_is_deletable, false) THEN
        RAISE EXCEPTION 'ERROR: This shift status is locked and
protected from deletion to preserve operational history.'
        USING ERRCODE = 'P0010';
    END IF;

    RETURN OLD;
END IF;

END;
$function$;

```

Добавление дневной записи со статусом забронирована:

```

CloudMonet=# insert into shift (type_id, status_id, shift_start) values (1, 1, '2026-05-20 12:30:00');
INSERT 0 1
CloudMonet=# select * from shift order by shift_id desc;

```

shift_id	shift_start	shift_end	type_id	status_id
5874	2026-05-20 12:30:00+03	2026-05-20 21:30:00+03	1	1

Пытаемся поставить дневную смену (норма 12:00) на 6 часов утра. База должна выкинуть ошибку **P0007**:

```

CloudMonet=# insert into shift (type_id, status_id, shift_start) values (1, 1, '2026-05-20 06:00:00');
ОШИБКА: ERROR: Shift start time is outside the allowed scheduling window.
КОНТЕКСТ: функция PL/pgSQL trg_shift(), строка 26, оператор RAISE

```

Добавляем дневную смену без указания конца смены. Триггер должен подставить значение `shift_start + duration`:

```

CloudMonet=# insert into shift (type_id, status_id, shift_start) values (1, 1, '2026-05-20 10:00:00');
INSERT 0 1
CloudMonet=# select * from shift order by shift_id desc;

```

shift_id	shift_start	shift_end	type_id	status_id
5876	2026-05-20 10:00:00+03	2026-05-20 19:00:00+03	1	1

Перевод смены с ID 5876 из состояния 1 (**ordered**) в ID 2 (**waiting**).

Массив разрешенных переходов для **ordered** — **{2}**, так что запрос выполнится успешно.

```

CloudMonet=# update v1.shift set status_id = 2 where shift_id = 5876;
UPDATE 1
CloudMonet=# select * from shift order by shift_id desc;

```

shift_id	shift_start	shift_end	type_id	status_id
5876	2026-05-20 10:00:00+03	2026-05-20 19:00:00+03	1	2

Пытаемся перевести смену из статуса `waiting` (ID 2) сразу в `closed` (ID 4), минуя работу. Разрешены только `{3, 5}`. База должна выкинуть ошибку `P0008`:

```
CloudMonet=# update v1.shift set status_id = 4 where shift_id = 5876;
ОШИБКА: ERROR: Requested status change is blocked by lifecycle rules.
КОНТЕКСТ: функция PL/pgSQL trg_shift(), строка 49, оператор RAISE
```

Переведем в `waiting`:

```
CloudMonet=# update v1.shift set status_id = 2 where shift_id = 5876;
UPDATE 1
CloudMonet=# select * from shift order by shift_id desc;
 shift_id |      shift_start      |      shift_end      | type_id | status_id
-----+-----+-----+-----+-----+
    5876 | 2026-05-20 10:00:00+03 | 2026-05-20 19:00:00+03 |       1 |         2
```

Переводим смену в статус `in-progress` (ID 3). Этот статус требует проверки времени (`requires_active_time = true`). Поскольку дата нашей смены в тесте указана как `2026-05-20`, а текущее реальное время на сервере другое, база должна выкинуть ошибку `P0009`:

```
CloudMonet=# update v1.shift set status_id = 3 where shift_id = 5876;
ОШИБКА: ERROR: Cannot change status because current real time is outside shift boundaries.
КОНТЕКСТ: функция PL/pgSQL trg_shift(), строка 55, оператор RAISE
```

Сначала создав новую запись напрямую в статусе `closed` (ID 4), пытаемся удалить её. База должна выкинуть ошибку `P0010`.

```
CloudMonet=# delete from shift where shift_id = 5877;
ОШИБКА: ERROR: This shift status is locked and protected from deletion to preserve operational history.
КОНТЕКСТ: функция PL/pgSQL trg_shift(), строка 68, оператор RAISE
```

3.4. Категории

Для сущности «Категории должностей» (`v1.category`) на начальном этапе проектирования было принято решение о внедрении логического флага `is_active` для обеспечения мягкого вывода департаментов из эксплуатации. Однако в ходе последующего анализа бизнес-модели данный флаг был удален как избыточный, поскольку структура ресторанных отделов статична, а наличие поля привело бы к усложнению SQL-запросов на бэкенде. В финальной схеме ограничение уникальности было перенесено на системный код (`category_code`), который был строго типизирован через `ENUM` в нижнем регистре. Также было ликвидировано ошибочное ограничение `fk_payment_type`, и был

внедрен триггер с регулярными выражениями (**REGEXP_REPLACE**) для автоматического удаления мусорных пробелов на входе.

Скрипт триггера:

```
CREATE OR REPLACE FUNCTION v1.trg_category()
  RETURNS trigger
  LANGUAGE plpgsql
  SET search_path TO 'v1'
AS $function$
DECLARE
  v_has_positions BOOLEAN;
BEGIN
  IF TG_OP IN ('INSERT', 'UPDATE') THEN
    NEW.name_en := TRIM(REGEXP_REPLACE(NEW.name_en, '\s+', ' ',
    'g'));
    NEW.name_ru := TRIM(REGEXP_REPLACE(NEW.name_ru, '\s+', ' ',
    'g'));

    IF TG_OP = 'UPDATE' AND OLD.is_active = true AND NEW.is_active =
false THEN
      IF EXISTS (SELECT 1 FROM v1.position WHERE category_id =
NEW.category_id AND is_active = true) THEN
        RAISE EXCEPTION 'ERROR: Cannot deactivate category while
it contains active positions.' USING ERRCODE = 'P0201';
      END IF;
    END IF;
    RETURN NEW;
  END IF;

  IF TG_OP = 'DELETE' THEN
    SELECT EXISTS (SELECT 1 FROM v1.position WHERE category_id =
OLD.category_id) INTO v_has_positions;
    IF v_has_positions THEN
      RAISE EXCEPTION 'ERROR: This category contains positions and
cannot be deleted. Use soft-delete (is_active = false).' USING ERRCODE =
'P0202';
    END IF;
    RETURN OLD;
  END IF;
END;
$function$;

CREATE TRIGGER trg_category_check
BEFORE INSERT OR UPDATE OR DELETE ON v1.category
FOR EACH ROW EXECUTE FUNCTION v1.trg_category();
```

Проверка сжатия пробелов на примере вставки текстовой строки внутри транзакции, которая при конфликте с уникальностью кода категории должна менять только ее названия на русском и английском :

```
CloudMonet=# INSERT INTO v1.category (category_code, name_en, name_ru, requires_med_book)
CloudMonet=# VALUES ((SELECT MIN(category_code) FROM v1.category), ' Kitchen Staff ', ' Кухонная Служба ', true)
CloudMonet=# ON CONFLICT (category_code) DO UPDATE
CloudMonet=# SET name_en = EXCLUDED.name_en, name_ru = EXCLUDED.name_ru;
INSERT 0 1
CloudMonet=#
CloudMonet=# SELECT name_en, name_ru FROM v1.category WHERE category_code = (SELECT MIN(category_code) FROM v1.category);
 name_en | name_ru
-----
Kitchen Staff | Кухонная Служба
(1 строка)
```

Запрет на физическое удаление категории со связанными должностями (Ошибка P0202) :

```
CloudMonet=# DELETE FROM v1.category
CloudMonet=# WHERE category_id = (SELECT category_id FROM v1.position LIMIT 1);
ОШИБКА: ERROR: This category contains positions and cannot be deleted.
КОНТЕКСТ: функция PL/pgSQL trg_category(), строка 14, оператор RAISE
```

3.5. Триггер перед сохранением должности

Для сущности «Должности» (`v1.position`) флаг `is_active` первоначально также планировался к внедрению для управления вакансиями, но затем был исключен, так как обеспечение историчности данных было признано более эффективным на уровне таблиц персонала. В ходе рефакторинга были упразднены некорректные `CHECK`-ограничения `fk_work_format` и `fk_payment_type`, а сами поля были переведены на строгие перечисления `ENUM` в формате `snake_case`. Бизнес-логика соответствия зарплат и грейдов (`base_salary` и `rank_level`) была запечатана на уровне СУБД через триггер, предотвращающий финансовые нарушения. Дополнительно в логику триггера была добавлена нормализация строк, с помощью которой названия приводятся к формату `INITCAP`, а лишние пробелы — схлопываются.

Содержание таблицы и ее метаданные:

```
CloudMonet=# select * from position;
```

position_id	category_id	payment_type	work_format	base_salary	rank_level	position_code	name_en	name_ru
16	3	salary	full_time	200000.0000	150	restaurant_manager	General Manager	Управляющий рестораном
17	3	salary	full_time	90000.0000	90	administrator	Shift Administrator	Администратор
21	3	salary	flexible	95000.0000	85	marketer	Marketing Specialist	Маркетолог
19	3	salary	remote	80000.0000	80	accountant	Accountant	Бухгалтер
18	3	salary	full_time	85000.0000	75	purchasing_manager	Purchasing Manager	Менеджер по закупкам
20	3	salary	full_time	75000.0000	70	hr_manager	HR Manager	Менеджер по персоналу
1	1	salary	full_time	150000.0000	100	chef	Head Chef	Шеф-повар
2	1	salary	full_time	100000.0000	80	sous_chef	Sous Chef	Су-шеф
8	1	hourly	full_time	85000.0000	55	sushist	Sushi Chef	Сушист
5	1	hourly	full_time	80000.0000	50	pastry_chef	Pastry Chef	Повар-кондитер
3	1	hourly	full_time	75000.0000	40	hot_dishes_cook	Hot Line Cook	Повар горячего цеха
7	1	hourly	full_time	75000.0000	40	butcher	Butcher	Мясник
4	1	hourly	full_time	70000.0000	35	cold_dishes_cook	Cold Line Cook	Повар холодного цеха
6	1	hourly	part_time	45000.0000	15	kitchen_helper	Kitchen Utility	Кухонный работник
9	2	hourly	full_time	65000.0000	60	head_waiter	Head Waiter	Старший официант
12	2	hourly	full_time	60000.0000	50	bartender	Bartender	Бармен
13	2	commission	full_time	55000.0000	45	hostess	Hostess	Хостес
14	2	hourly	full_time	50000.0000	40	cashier	Cashier	Кассир
10	2	hourly	full_time	45000.0000	30	waiter	Waiter	Официант
15	2	hourly	part_time	35000.0000	20	runner	Runner	Ранер
11	2	hourly	part_time	30000.0000	10	trainee_waiter	Trainee Waiter	Официант (стажёр)

(21 строка)

```
Индексы:
"pk_position" PRIMARY KEY, btree (position_id)
"idx_pos_rank_lvl" btree (rank_level)
"uq_position_code" UNIQUE CONSTRAINT, btree (position_code)
Ограничения внешнего ключа:
"fk_category" FOREIGN KEY (category_id) REFERENCES category(category_id) ON UPDATE CASCADE ON DELETE RESTRICT NOT VALID
Ссылки извне:
TABLE "worker" CONSTRAINT "fk_position" FOREIGN KEY (position_id) REFERENCES "position"(position_id) ON UPDATE CASCADE ON DELETE RESTRICT NOT VALID
TABLE "worker_shift" CONSTRAINT "fk_ws_position_in_shift" FOREIGN KEY (position_in_shift) REFERENCES "position"(position_id) NOT VALID
```

Скрипт триггера:

```
CREATE OR REPLACE FUNCTION v1.trg_position_hierarchy_check()
RETURNS trigger
LANGUAGE plpgsql
SET search_path TO 'v1'
AS $$
BEGIN
    NEW.name_en := INITCAP(TRIM(REGEXP_REPLACE(NEW.name_en, '\s+', ' ',
'g')));
    NEW.name_ru := INITCAP(TRIM(REGEXP_REPLACE(NEW.name_ru, '\s+', ' ',
'g')));

    IF NEW.position_code IS NULL THEN
        RAISE EXCEPTION 'ERROR: position_code is required.' USING
ERRCODE = 'P0104';
    END IF;
    IF TG_OP = 'UPDATE' AND OLD.position_code <> NEW.position_code THEN
        RAISE EXCEPTION 'ERROR: Changing immutable position_code is
prohibited.' USING ERRCODE = 'P0105';
    END IF;
    IF NEW.base_salary <= 0 THEN
        RAISE EXCEPTION 'ERROR: Base salary must be greater than zero.'
USING ERRCODE = 'P0101';
    END IF;
    IF NEW.rank_level <= 0 THEN
        RAISE EXCEPTION 'ERROR: Rank level must be a positive number.'
USING ERRCODE = 'P0106';
    END IF;

    IF EXISTS (
```

```

        SELECT 1 FROM v1.position
        WHERE category_id = NEW.category_id
              AND position_id <> COALESCE(NEW.position_id, -1)
              AND rank_level < NEW.rank_level
              AND base_salary > NEW.base_salary
    ) THEN
        RAISE EXCEPTION 'ERROR: Financial hierarchy violation. Lower
rank cannot have a higher salary.' USING ERRCODE = 'P0107';
    END IF;

    IF EXISTS (
        SELECT 1 FROM v1.position
        WHERE category_id = NEW.category_id
              AND position_id <> COALESCE(NEW.position_id, -1)
              AND rank_level > NEW.rank_level
              AND base_salary < NEW.base_salary
    ) THEN
        RAISE EXCEPTION 'ERROR: Financial hierarchy violation. Higher
rank cannot have a lower salary.' USING ERRCODE = 'P0108';
    END IF;

    RETURN NEW;
END;
$$;

create trigger trg_position_before_save before update or insert on
position for each row execute function trg_position_hierarchy_check();

```

Попытка добавить должность существующей должности помощника повара из категории кухни(ID 1) с рангом 15 претендующего на зарплату 75 000 с последующим из-за конфликта обновлением имен на русском и английских языках. Но при обновлении оказывается, что в базе есть кухонный работник с более высоким рангом 30, который получает меньше – 70 000. Это нарушение правила "Высокий ранг не может получать меньше низкого"!» И триггер абсолютно справедливо пробрасывает ошибку.

Скрипт транзакции:

```

BEGIN;

INSERT INTO v1.position (category_id, position_code, payment_type,
work_format, name_en, name_ru, base_salary, rank_level)
VALUES (
    1,

```



```

        'kitchen_helper',
        'salary',
        'full_time',
        ' junior cook ',
        ' младший повар ',
        75000,
        15
    )
ON CONFLICT (position_code) DO UPDATE
SET name_en = EXCLUDED.name_en, name_ru = EXCLUDED.name_ru;

ROLLBACK;

```

```

CloudMonet-## VALUES (
CloudMonet(##      1,
CloudMonet(##      'kitchen_helper',
CloudMonet(##      'salary',
CloudMonet(##      'full_time',
CloudMonet(##      ' junior cook ',
CloudMonet(##      ' младший повар ',
CloudMonet(##      75000.00,
CloudMonet(##      15
CloudMonet(## )
CloudMonet-## ON CONFLICT (position_code) DO UPDATE
CloudMonet-## SET name_en = EXCLUDED.name_en, name_ru = EXCLUDED.name_ru;
ОШИБКА: ERROR: Financial hierarchy violation. Higher rank cannot have a lower salary.
КОНТЕКСТ: функция PL/pgSQL trg_position_hierarchy_check(), строка 36, оператор RAISE
CloudMonet=!#

```

Пример с тестовыми данные, которые не ломают экономику кухни:

```

CloudMonet-## VALUES (
CloudMonet(##      1,
CloudMonet(##      'kitchen_helper',
CloudMonet(##      'salary',
CloudMonet(##      'full_time',
CloudMonet(##      ' junior cook ',
CloudMonet(##      ' младший повар ',
CloudMonet(##      69000.00,
CloudMonet(##      15
CloudMonet(## )
CloudMonet-## ON CONFLICT (position_code) DO UPDATE
CloudMonet-## SET name_en = EXCLUDED.name_en, name_ru = EXCLUDED.name_ru;
INSERT 0 1
CloudMonet=##
CloudMonet=## SELECT name_en, name_ru
CloudMonet-## FROM v1.position
CloudMonet-## WHERE rank_level=15;
   name_en   |   name_ru
-----+-----
 Junior Cook | Младший Повар
(1 строка)

```

3.6. Триггер перед удалением должности

На этапе выше лабораторной работы пришла замечательная идея, о том что стоит разделять триггеры на BEFORE SAVE и BEFORE DELETE, чтобы база не тратила ресурсы на сканирование таблицы сотрудников перед INSERT и UPDATE, а также повысилась чистота кода, поскольку внутри функций была устранена мешанина из условных операторов проверки типа операции (**IF TG_OP = ...**). То есть первая функция стала отвечать исключительно за экономическую валидацию и нормализацию строк, а вторая – за безопасность удаления и ссылочную целостность данных, что обеспечит прозрачность структуры для дальнейшей интеграции с бэкендом.

```
CREATE OR REPLACE FUNCTION v1.trg_position_delete_check()
  RETURNS trigger
  LANGUAGE plpgsql
  SET search_path TO 'v1'
AS $function$
DECLARE
  v_has_employees BOOLEAN;
BEGIN
  SELECT EXISTS (
    SELECT 1 FROM worker WHERE position_id = OLD.position_id
  ) INTO v_has_employees;

  IF v_has_employees THEN
    RAISE EXCEPTION 'ERROR: This position is currently assigned to
operational workers and cannot be deleted.'
    USING ERRCODE = 'P0102';
  END IF;

  RETURN OLD;
END;
$function$;

CREATE TRIGGER trg_position_before_delete
  BEFORE DELETE ON v1.position
  FOR EACH ROW EXECUTE FUNCTION v1.trg_position_delete_check();
```

Попытка удаления должности официанта:

```
CloudMonet=# delete from position where position_id = 10;
ОШИБКА:  ERROR: This position is currently assigned to operational workers and cannot be deleted.
КОНТЕКСТ:  функция PL/pgSQL trg_position_delete_check(), строка 10, оператор RAISE
```

3.7. Аудиты

Для отслеживания всех изменений в базе данных была создана единая система логов. Она позволяет в любой момент посмотреть, кто, когда и что именно изменил.

Все логи собираются в одну таблицу. Благодаря типу данных `jsonb`, в ней можно сохранять строки абсолютно любого формата из любых таблиц, не меняя структуру самой таблицы аудита.

Что хранится в таблице:

- **table_name** и **operation** — название таблицы и тип действия (`INSERT`, `UPDATE`, `DELETE`).
- **row_id** — ID измененной строки для быстрого поиска.
- **old_data** и **new_data** — точные слепки строки «до» и «после» изменений в формате JSON.
- **changed_by** — имя пользователя базы данных, который сделал изменения.
- **changed_at** — точное время операции.

Чтобы не писать отдельный код для каждой таблицы, была создана одна общая функция `v1.trg_generic_audit_log()`. Она сама определяет, с какой таблицей работает и какое действие происходит.

Она работает в режиме `SECURITY DEFINER` — это значит, что лог запишется в любом случае, даже если у обычного пользователя нет прав на прямую вставку в таблицу аудита.

Исходный код универсальной функции:

```
CREATE OR REPLACE FUNCTION v1.trg_generic_audit()  
  RETURNS trigger  
  LANGUAGE plpgsql  
  SECURITY DEFINER  
  AS $function$  
  begin  
    insert into v1.audit_logs (table_name, operation, row_id, old_data,  
    new_data)  
    values (
```

```

TG_TABLE_SCHEMA || '.' || TG_TABLE_NAME,
TG_OP,
-- TODO: After the planned refactoring and renaming of all
primary keys
-- (position_id, category_id, etc.) to a unified "id" standard,
-- this block will shrink down to: (case when TG_OP = 'DELETE'
then to_jsonb(old) else to_jsonb(new) end ->> 'id')::integer
coalesce(
    (case when TG_OP = 'DELETE' then to_jsonb(old) else
to_jsonb(new) end ->> 'position_id')::integer,
    (case when TG_OP = 'DELETE' then to_jsonb(old) else
to_jsonb(new) end ->> 'id')::integer
),
case when TG_OP in ('UPDATE', 'DELETE') then to_jsonb(old) else
null end,
case when TG_OP in ('INSERT', 'UPDATE') then to_jsonb(new) else
null end
);

if TG_OP = 'DELETE' then return old; else return new; end if;
end;
$function$;

```

В коде универсального триггера заложена обратная совместимость с текущей схемой данных таблицы position, а также оставлен технический комментарий (TODO) для запланированного перехода на единый стандарт именования первичных ключей (**id**), что в будущем позволит сократить и упростить логику извлечения идентификаторов строк.

Для активации аудита функция связывается с целевыми таблицами с помощью триггеров уровня **AFTER**. Это гарантирует, что в аудит попадают **только успешно завершённые и валидные операции**, прошедшие все бизнес-проверки (включая контроли уровня **BEFORE**).

Пример создания триггера для таблицы должностей:

```

CREATE OR REPLACE TRIGGER trg_audit_position
AFTER INSERT OR UPDATE OR DELETE ON v1.position
FOR EACH ROW
EXECUTE FUNCTION v1.trg_generic_audit_log();

```

Пример транзакции:

```

BEGIN;

UPDATE v1.position
SET name_ru = '    Шеф-Повар    Новый    '
WHERE position_id = 1;

SELECT table_name, operation, row_id, old_data->>'name_ru' as was_name,
new_data->>'name_ru' as now_name, changed_by, changed_at
FROM v1.audit_logs
ORDER BY id DESC
LIMIT 1;

ROLLBACK;

```

Содержимое таблицы с логами ло отката:

table_name	operation	row_id	was_name	now_name	changed_by	changed_at
v1.position	UPDATE	1	Шеф-повар	Шеф-Повар Новый	postgres	2026-05-20 04:31:30.592396+03

(1 строка)

Выводы

В процессе выполнения данной работы стало очевидно, что проектирование базы данных на стадии обучения — это живой, итеративный процесс, потребовавший колоссального количества изменений и адаптаций схемы «на лету». Это наглядно прослеживается по всему отчету: структура таблиц в начале проектирования, в ходе выполнения работы и в её финальной точке кардинально отличаются. Столкнувшись на практике с необходимостью переноса локализации, исправлением логических ловушек в финансовой иерархии и созданием универсального аудита, мне пришлось полностью переосмыслить архитектуру данных, что привело к решению о будущем переименовании всех первичных ключей в единый стандарт id.